ADVANCED AI MEDICAL INTELLIGENCE PLATFORM

End-to-End Medical Image Analysis, Explainable AI & AI-Assisted Research Reporting

Project Report

Technologies: Python • PyTorch • DenseNet121 • Grad-CAM • FastAPI • Streamlit • SQLite/SQLAlchemy • Ollama/Llama 3.2 • Docker

Model: DenseNet121
Dataset: NIH Chest X-ray (simplified five-class research task)
Classes: Normal, Pneumonia, Atelectasis, Cardiomegaly, Effusion

Prepared as a reproducible local/Docker research prototype

Abstract

The Advanced AI Medical Intelligence Platform is an end-to-end research prototype for chest X-ray image analysis. The system combines a transfer-learning deep learning classifier based on DenseNet121 with Explainable AI using Grad-CAM, a FastAPI REST service, persistent prediction history using SQLite/SQLAlchemy, and an AI-assisted research report generator using an Ollama-hosted large language model. A Streamlit interface provides an accessible workflow for uploading an X-ray, viewing the prediction and confidence, inspecting the Grad-CAM visualization, reading the generated research report, and reviewing previous predictions. Docker packaging provides a reproducible execution path. The platform is designed for research and demonstration and is not a clinical diagnostic device.

Keywords

Medical Image Classification; DenseNet121; Transfer Learning; Grad-CAM; Explainable AI; FastAPI; Streamlit; LLM; Docker; Chest X-ray

1. Introduction

Medical imaging applications can benefit from machine learning models that assist with image classification while also providing interpretable evidence. A major limitation of a prediction-only system is that users may not understand why a model produced a particular result. This project addresses that limitation by combining classification with Grad-CAM visual explanations and an AI-assisted structured research report. The result is a complete application rather than an isolated machine learning notebook.

2. Objectives

The primary objectives are to: (1) classify selected chest X-ray images into five research classes; (2) expose the model through REST APIs; (3) generate Grad-CAM visual explanations; (4) store prediction history and model metadata; (5)

generate an AI-assisted research report from structured prediction output; (6) provide a Streamlit user interface; (7) package the application with Docker; and (8) document a reproducible local/Docker setup.

3. System Architecture / Project Flow

Input X-ray → Streamlit UI → FastAPI /predict → Image preprocessing → DenseNet121 inference → Prediction + confidence → Grad-CAM generation → SQLite/SQLAlchemy history → LLM report generation → API response → Streamlit visualization

The architecture separates presentation, API, model/XAI logic, persistence, and reporting responsibilities. Streamlit is the user-facing layer. FastAPI provides service endpoints. DenseNet121 performs inference. Grad-CAM provides visual interpretability. SQLite/SQLAlchemy stores prediction records. Ollama provides the local LLM interface for research-report generation. Docker packages the API runtime.

Architecture Components

Component	Purpose
Streamlit	Upload X-rays, display predictions, Grad-CAM, reports and history.
FastAPI	REST API and Swagger documentation.
DenseNet121	Five-class chest X-ray image classification.
Grad-CAM	Visual explanation of model attention.
SQLite + SQLAlchemy	Persistent prediction history and metadata.
Ollama / Llama 3.2	AI-assisted research report generation.
Docker	Reproducible API runtime and deployment packaging.

4. Dataset and Model

The project uses the NIH Chest X-ray dataset as the research image source. For this prototype, the task was simplified to five classes: Normal, Pneumonia, Atelectasis, Cardiomegaly and Effusion. The processed dataset was split into training, validation and test sets. The final training workflow used transfer learning with DenseNet121.

Training / Validation Metrics

Metric	Value
Training images	53,330
Validation images	11,428
Training loss	0.8400
Training accuracy	70.91%
Validation loss	0.8463
Validation accuracy	69.69%
Model	DenseNet121

Test Confusion Matrix

The recorded test confusion matrix (class order: Atelectasis, Cardiomegaly, Effusion, Normal, Pneumonia) was:

	Atelectasis	Cardiomegaly	Effusion	Normal	Pneumonia

Atelectasis	171	1	388	975	0
Cardiomegaly	13	2	67	244	0
Effusion	81	1	525	619	0
Normal	226	4	712	7205	0
Pneumonia	9	0	50	136	0

These metrics should be interpreted as prototype/research metrics. The class distribution is imbalanced and the confusion matrix shows substantial confusion among several classes, so accuracy alone should not be interpreted as clinical performance.

5. API Design

Endpoint	Purpose
GET /	API home/status information.
GET /health	Health check.
GET /model-info	Model information and metadata.
POST /predict	Upload an image and obtain prediction, confidence, Grad-CAM reference and report.
GET /gradcam/{filename}	Retrieve generated Grad-CAM image.
GET /predictions	Retrieve prediction history.

6. Explainable AI – Grad-CAM

Grad-CAM is used to generate a class-discriminative localization map from the convolutional feature representations. The heatmap is used to provide a visual indication of image regions that contributed to the model's output. In this prototype, the generated Grad-CAM image is exposed through the API and displayed by Streamlit.

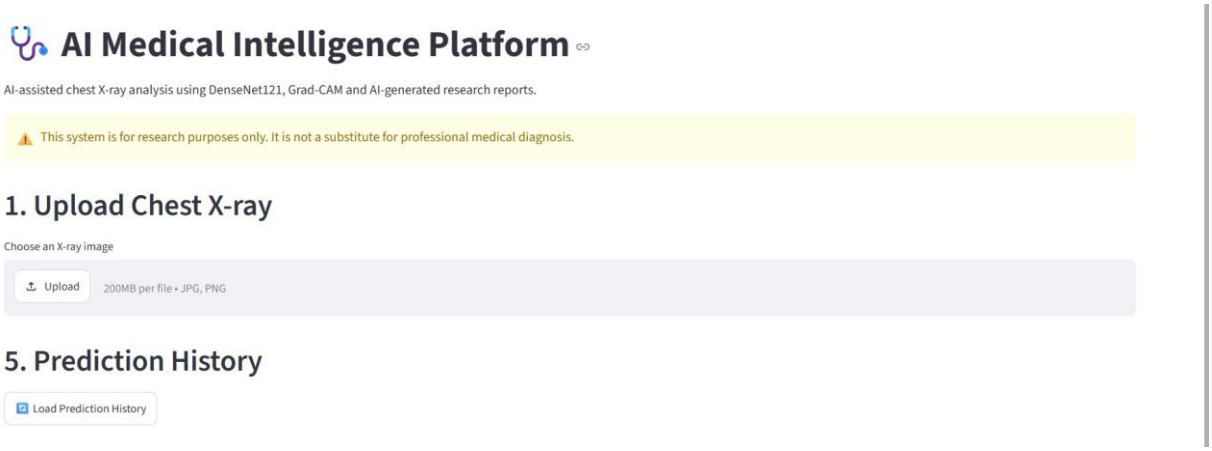


Figure 1. Streamlit Home / X-ray Upload Interface

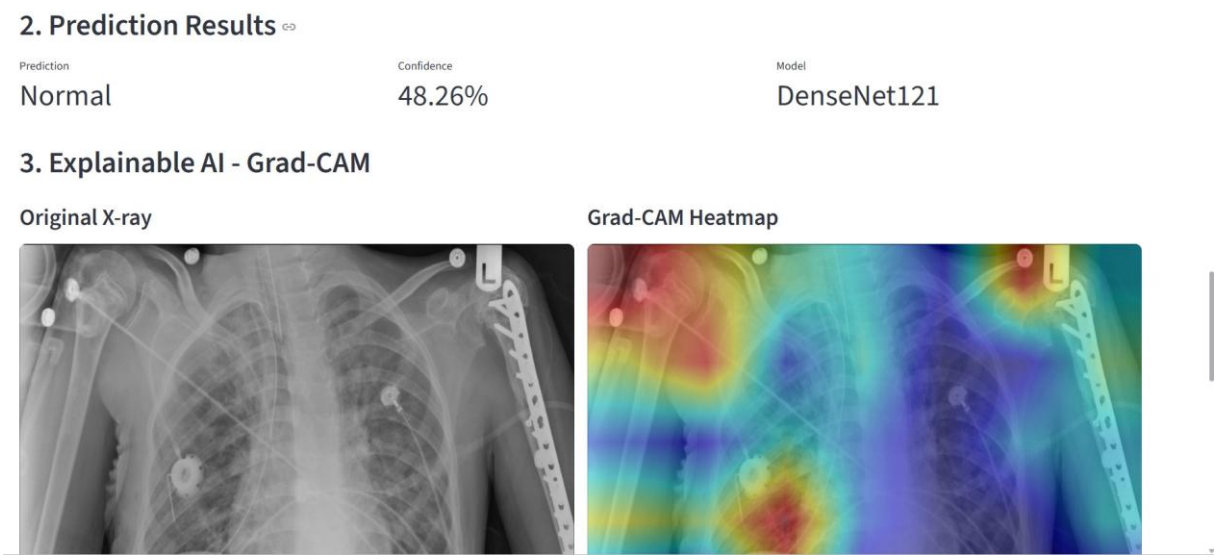


Figure 2. Prediction Result and Grad-CAM

7. AI-Assisted Research Report

The report-generation component receives structured information such as model name, prediction and confidence and asks a local LLM to produce a structured research-oriented interpretation. The interface displays the generated report together with limitations and a prominent disclaimer. The LLM output is an assistive text layer and does not replace professional medical interpretation.

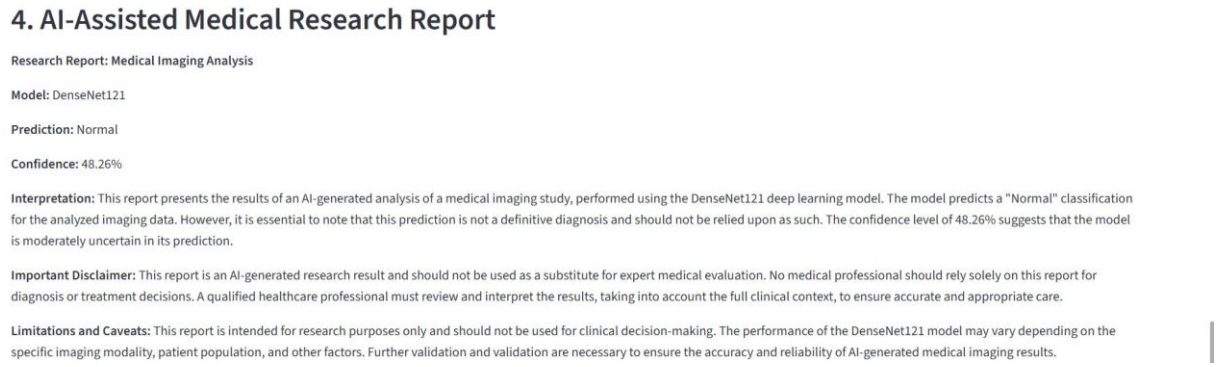


Figure 3. AI-Assisted Medical Research Report

8. Prediction History and Database

Prediction records are persisted using SQLite with SQLAlchemy models. The project stores information such as image reference, prediction, confidence, model/model version, Grad-CAM path and generated report. This enables users to review previous inference results from the Streamlit interface and provides a traceable research workflow.

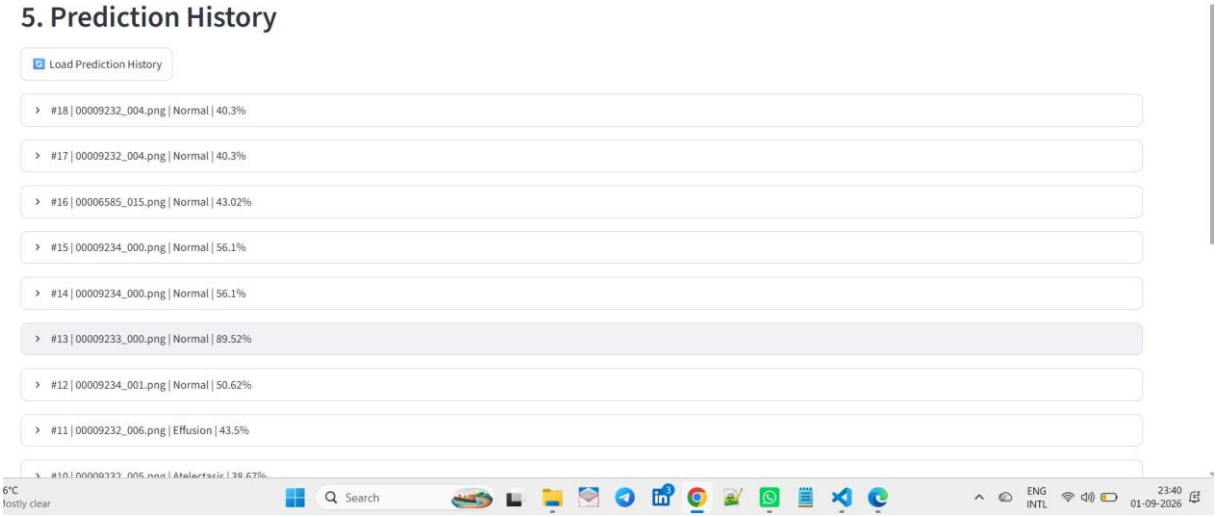


Figure 4. Prediction History

9. FastAPI Swagger Documentation

FastAPI automatically provides interactive OpenAPI/Swagger documentation at /docs. The captured interface demonstrates the implemented endpoints and makes the API straightforward to test locally.

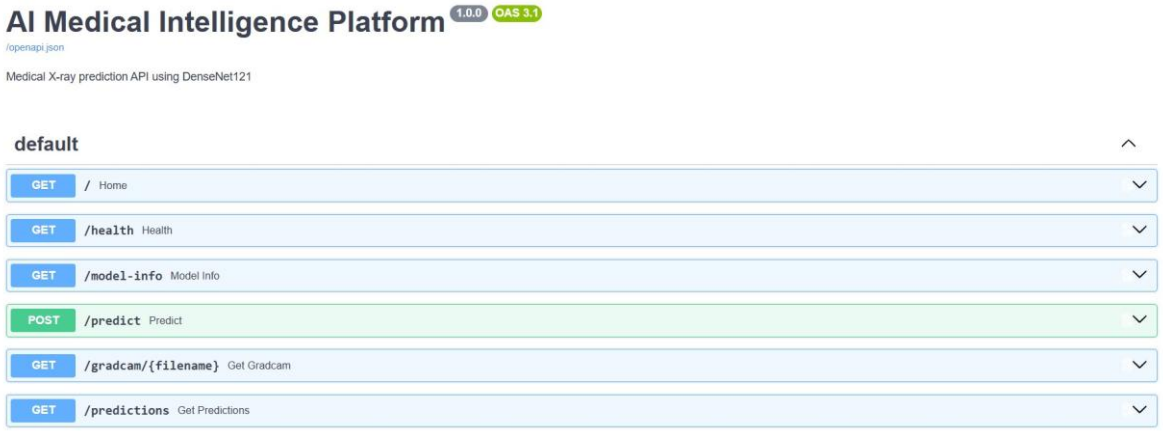


Figure 5. FastAPI Swagger /docs

10. Docker Deployment / Reproducibility

The API is packaged using a Python 3.11 slim base image. Dependencies are installed from requirements.txt and Uvicorn serves the FastAPI application on port 8000. The container was successfully running as **medical-ai-container** from image **medical-ai-api**, with port mapping **0.0.0.0:8000 → 8000/tcp**. Ollama is accessed from the container through the host gateway configuration used during testing.

Verified Docker evidence

Field	Observed value
Container	medical-ai-container
Image	medical-ai-api
Status	Up / running

Port	0.0.0.0:8000->8000/tcp
API server	Uvicorn / FastAPI

Example reproducible commands

```
docker build -t medical-ai-api .
docker run -d -p 8000:8000 --name medical-ai-container --add-host=host.docker.internal:host-gateway -
e OLLAMA_HOST=http://host.docker.internal:11434 medical-ai-api
```

11. Streamlit User Workflow

1. Start the API service. 2. Start Streamlit. 3. Upload a chest X-ray. 4. Submit the image for prediction. 5. Review prediction and confidence. 6. Inspect Grad-CAM. 7. Read the AI-assisted research report. 8. Review prediction history. The captured screenshots demonstrate these major flows.

12. Testing and Validation

Major application flows were exercised during development: image upload and prediction, Grad-CAM generation and retrieval, AI report generation, prediction history retrieval, FastAPI Swagger testing, and Docker container execution. The /predict endpoint was verified with a successful HTTP 200 response containing prediction, confidence, model information, Grad-CAM path and report.

13. Security and Repository Hygiene

A .gitignore file was added before the final GitHub push. It excludes the Python virtual environment, .env secrets, local database, generated Grad-CAM outputs, dataset directories, Python cache files and model weight files (*.pth). An .env.example file is retained as a configuration template. This prevents sensitive/local artifacts from being accidentally committed.

14. Limitations

This is a research/educational prototype and should not be used to make medical decisions. The dataset is imbalanced, and the recorded validation accuracy of 69.69% does not establish clinical safety or generalization. The displayed confidence is a model score, not a probability of disease confirmed by a clinician. Grad-CAM is an interpretability aid and should not be treated as a definitive anatomical explanation. LLM-generated reports may contain errors and require expert review.

15. Future Enhancements

Potential improvements include class balancing and targeted sampling, stronger evaluation using macro F1/AUROC and per-class sensitivity/specificity, calibration analysis, improved model training with more compute, authentication and authorization for production APIs, cloud deployment, automated CI/CD, model version management, richer audit logging, and stronger clinical validation with appropriately curated datasets and expert review.

16. Conclusion

The Advanced AI Medical Intelligence Platform demonstrates a complete AI application lifecycle: data preparation, deep learning inference, explainability, API serving, persistence, LLM-assisted reporting, interactive visualization, Docker packaging and GitHub-based source management. The implemented system is reproducible locally and through Docker and provides a strong foundation for further research and engineering improvements.

Appendix A – Key Project Structure

api/main.py — FastAPI application and REST endpoints
database/database.py — database connection
database/models.py — SQLAlchemy models
src/train.py — DenseNet121 training src/evaluate.py — evaluation and confusion matrix src/gradcam.py — Grad-CAM generation src/report_generator.py — AI-assisted report generation streamlit_app.py — Streamlit user interface Dockerfile — container image definition
requirements.txt — Python dependencies
.gitignore — repository hygiene and secret/artifact protection

Appendix B – Research Disclaimer

This project is intended for research, educational and software-engineering demonstration purposes only. It is not a medical device and does not provide a definitive diagnosis. All outputs, including model predictions, confidence scores, Grad-CAM visualizations and LLM-generated reports, must be reviewed by a qualified healthcare professional before any clinical interpretation or action.